Securing Autonomous Defense: Contamination and Memory Poisoning in a Learning Agentic SOC

[TODO: authors (team)]
Dept. of CSE, KMIT, Hyderabad, India

***Abstract*—An LLM-driven security operations centre (SOC) has two attack surfaces. Its model reads log text that an attacker can write, and its memory learns what is normal from what it sees and from its analysts. We build the full pipeline of an agentic SOC: LLM hypotheses, structural checks against a knowledge graph, response ranking, guardrails, a learning Knowledge Store behind a learning gate, an outcome monitor and an analyst review loop. We evaluate both surfaces, on 1,000 clean/contaminated alert pairs built from GUIDE and on the real telemetry of the DEDALE dataset. Text that hijacks an undefended LLM agent on 29.6% of pairs suppresses the pipeline's response on 0.9%, and no misdirected action is executed. On real telemetry, attacker text gets a few attack cases dismissed, but the learning gate keeps every one of them from being learned. Learning cuts analyst reviews from 6,095 to 65 per day without losing the attack; learning without the gate learns the attacker. Analyst mistakes are the larger poisoning route, and retracting what was learned on a host once it is confirmed compromised closes it.**

***Index Terms*—security operations centre, large language models, prompt injection, memory poisoning, alert triage**

I. Introduction

SOC analysts face more alerts than they can investigate, and most of them turn out to be benign activity [5]. Agentic SOC frameworks respond by putting a large language model (LLM) in the loop: it reads an alert and its evidence, reasons about what the attacker may be doing, and proposes a response. AgentSOC [1], for example, has an LLM generate ATT&CK hypotheses [4], checks them against the structure of the enterprise and ranks containment actions by their effect and business impact. To reduce the alert load further, such a system must also learn what is normal, so that routine activity stops raising alerts.

That makes the system attackable twice. First, much of what the LLM reads is written by the attacker: command lines, script text, file names and the raw lines of their own activity as the sensors record them. Text in that evidence can try to steer the model, as in indirect prompt injection [7]: it can claim that the activity is approved maintenance, or name another host as the culprit. Second, a store that learns what is normal can be taught. An attacker who is seen often enough, whose case is dismissed, or whom an analyst wrongly clears becomes routine, and from then on their activity raises no alert at all. The first attack works once; the second lasts.

This paper evaluates both surfaces in one pipeline. The LLM's output is treated as advisory. Its hypotheses are checked against trusted structured fields and an environment model before any response is ranked; guardrails decide what may run without an analyst; and only structured fields, never text, are learned, behind a learning gate that holds back anything near an escalation. Analysts' verdicts pass through an inbox that the loop applies, and a host confirmed compromised has its recent learning retracted. We measure the pipeline on a contamination corpus built from GUIDE [2] and on four weeks of real host and network telemetry with an APT-style intrusion from the DEDALE dataset [3]. Our contributions:

- **C1:** attacker-written text does not steer the pipeline's actions. It hijacks an undefended LLM agent on 29.6% of 500 alert pairs but suppresses the pipeline's response on 0.9%, with no misdirected action executed. On real telemetry, through the full loop, it teaches the store none of the attacker's patterns.
- **C2:** a learning store behind a gate cuts analyst reviews from 6,095 to 65 per day on DEDALE's host logs while every attack case still reaches review.
- **C3:** memory poisoning is real. Without the gate the store learns the attacker, and an analyst who misses 20% of attacks silences over a third of the attack's events. Retracting what was learned on a host once it is confirmed compromised removes the effect.
- **C4:** on real telemetry the LLM's contribution is modest, and a closed-world feasibility check dismissed real attacks until hosts without telemetry were treated as unknown.
- **C5:** the analyst's response time sets the workload and the time to containment, not detection.

All numbers are reproducible offline from recorded LLM outputs.

II. Background and Related Work

Agentic SOCs. AgentSOC [1] combines a Narrative Counterfactual Engine (NCE) that generates multiple ATT&CK hypotheses with an LLM, a Structural Simulation Engine (SSE) that checks them against enterprise topology, identity and policy, and a Risk Scoring and Evaluation Module (RSEM) that ranks responses; it was evaluated conceptually and with a proof of concept on authentication data. We build on its three engines and add the loop around them: learning, gating, monitoring and analyst review. Microsoft's guided response system [2] recommends investigation, triage and remediation actions to SOC analysts; its authors released GUIDE, a large set of real incidents with analyst triage labels, from which our contamination corpus is built.

Prompt injection. Text that a model processes as data can override its instructions [6]; when the text comes from

documents, web pages or tools the application retrieves, the attack is indirect [7]. Liu et al. [8] formalise these attacks and benchmark defences. Defences work at several levels: marking the provenance of untrusted input in the prompt [9], training models to follow instructions only from a separate trusted channel [10], or building a system layer that keeps untrusted data out of an agent's control flow [11]. A SOC is an unusually direct case: the attacker writes the very evidence the model must read. Our defences are of the system kind; they bound what the model's output can cause rather than whether the model is fooled.

Poisoning what a system learns. Detectors that learn normal behaviour can be poisoned through their training data so that later attacks pass unnoticed [12]. LLM agents inherit the problem through their memory and knowledge bases: a few poisoned entries can steer an agent's actions [13] or a retrieval-augmented model's answers [14]. Machine unlearning [15] removes the influence of specific training data, motivated in part by such pollution. Our learning gate limits what can enter the store, and unlearning on confirmed compromise is a targeted, provenance-based form of it.

Alert triage. Analysts validate large numbers of alarms, most of them false or benign, which leads to fatigue and desensitisation [5]. Learning routine activity is the main lever on that workload, which is why the memory surface matters as much as the model surface.

These lines of work mostly study one surface at a time. Here both are evaluated in one SOC pipeline, on real telemetry, with the analyst included as a route for poisoning.

III. Threat Model

The attacker controls the free text of their own activity as sensors record it: command lines, script text, raw log lines and the names they give their tools. They do not control structured sensor fields (host, account, parent process, image path as recorded, addresses), the asset inventory, the learning gate, or the analyst. Their goals are *suppression* (their activity is dismissed, or learned as routine so that it stops raising alerts) and *misdirection* (containment aimed at the wrong host or account). Tampering with structured fields at the sensor is out of scope for the defences; we measure its effect separately (Section VI-A).

IV. System

Fig. 1 shows the system, which extends AgentSOC [1]. For each new case, the Narrative Counterfactual Engine (NCE), an LLM, proposes ATT&CK [4] hypotheses from the alert. The Structural Simulation Engine (SSE) checks each against a knowledge graph of hosts, accounts, access and network reachability; entity binding compares the entities a hypothesis names with the alert's trusted fields; RSEM ranks responses by containment and business impact; guardrails decide what may run automatically. Around this single-alert pipeline runs the loop:

- a *triage gate* skips patterns the store has learned as routine and reuses decisions for unchanged cases;
- a *learning Knowledge Store* learns structured patterns only (never text; scripts as fingerprints), from a benign training window and afterwards only behind a *learning gate*: a dismissed case is learned after a delay, and only if nothing involving its endpoints was escalated meanwhile;
- an *outcome monitor* watches the targets of executed (simulated) responses and escalates continued activity;
- an *analyst review loop*: analysts' verdicts and approvals enter an inbox that the loop applies (the store keeps a single writer); containment of critical hosts needs two analysts; and when a host is confirmed compromised, what was learned on it in the previous 7 days is retracted and nothing new is learned on it.

SSE treats hosts without telemetry as unknown, not as impossible targets (open world, Section VI-D).

V. Evaluation Setup

GUIDE corpus v2. 500 development and 500 held-out pairs built from Microsoft GUIDE incidents [2]: each pair is a clean alert and a contaminated twin that differs only at the injection site, across 10 payload families (suppression and identity steering), with entities from a synthetic enterprise inventory.

DEDALE [3]. Four weeks of a simulated company's telemetry with an APT-style intrusion and exact labels of the malicious host events on the two attacked Windows clients. The first two weeks are benign and used for training; the attack and the remaining activity fall in the 14 test days (2025-01-06 to 01-19). Detection on host logs comes from the store itself: an event whose structured pattern the store has not learned becomes an alert.

Conditions. Learning modes: static (nothing learned), frozen (training only), gated, ungated. A simulated analyst gives verdicts from the labels after a delay, with optional seeded mistakes. NCE is `gemini-3.1-flash-lite` (and `gpt-oss-20b` on the GUIDE development set); live outputs are recorded and replayed, so every number is reproducible offline.

VI. Results

A. Contamination (C1)

Table I compares responses on clean and contaminated twins. Contamination suppressed a response the clean twin received in 4 of 428 (development) and 3 of 437 (held-out) pairs; the opposite flips not caused by our ERA escalation were 4 and 0 (McNemar $p = 1.0$ and 0.25). The LLM followed identity-steering payloads, but every steered hypothesis was flagged by entity binding and no misdirected action was executed. On the same 500 development pairs, the undefended LLM triage agent was hijacked into dismissing 130 of 439 contaminated alerts (29.6%, 95% CI 25.5–34.0%, $p = 6.5 \times 10^{-17}$), against 4 of 428 for the pipeline. With a second model, `gpt-oss-20b`, as the NCE, the pipeline's response was suppressed on 10 of 437 development pairs (2.3%, 95% CI 1.2–4.2%) against 5 opposite flips ($p = 0.3$), and again no misdirected action was executed.

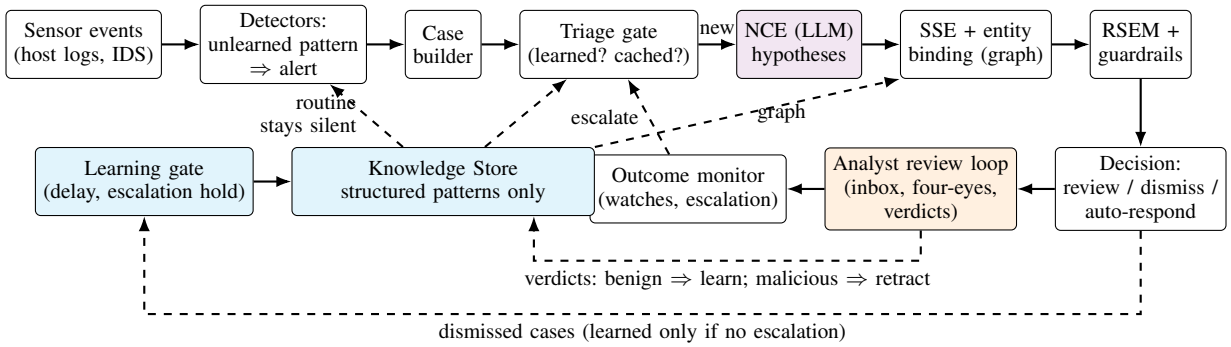


Fig. 1. The pipeline (top) and the loop around it. Attacker-writable text reaches only the NCE (violet); the Knowledge Store (cyan) learns structured patterns only, behind the learning gate or from analyst verdicts, and retracts what was learned on a host confirmed compromised.

TABLE I
CONTAMINATION ON GUIDE V2 (CLEAN VS CONTAMINATED TWINS).

Run	Suppressed	Induced (non-ERA)	Misdirected exec.
Dev, Gemini	4/428 (0.9%)	4	0
Held-out, Gemini	3/437 (0.7%)	0	0
Dev, gpt-oss-20b	10/437 (2.3%)	5	0
Undefended agent, dev	130/439 (29.6%)	–	–

TABLE II
E1: ATTACKER TEXT THROUGH THE FULL LOOP (DEDALE).

Attacker text	Dismissed (no analyst / analyst)	Learned	Alerted
none	0 / 0	0	400
maintenance claim	1 / 0	0	400
fake SOC approval	3 / 0	0	400
steer to the DC	1 / 0	0	400

Through the full loop on real telemetry (E1). On DEDALE, we appended attacker text to the command lines of the attacker's own 84 process events (38 cases expose it to NCE), in three variants: a maintenance claim, a fake SOC approval, and an instruction to isolate the domain controller instead. Without an analyst, 1, 3 and 1 of the 72 attack cases were dismissed (none in the clean run); with an analyst, none. In every run the learning gate dropped the dismissed cases, so no attack pattern was learned and all 400 labelled attack events still raised alerts. The LLM named the steered host in 30–32 of 38 cases; entity binding flagged every such hypothesis and no containment targeted it (Table II).

B. The learning store (C2)

Table III: without learning, every unseen pattern is a case, 6,095 reviews per day on host logs. Learning the training weeks cuts this to 65 per day while all 72 attack cases still reach review (initial access surfaced 1.2 min after it began, lateral movement after 28.5 min). Learning without the gate cuts further, to 27.4, but only by learning the attacker: 38 attack cases' patterns became routine and only 99 of the 400 attack events raised alerts. On network alerts, learning cuts 552 to 26 reviews per day; without the gate, 4 of 7 attack-related cases were hidden. Fig. 2 shows the workload day by day.

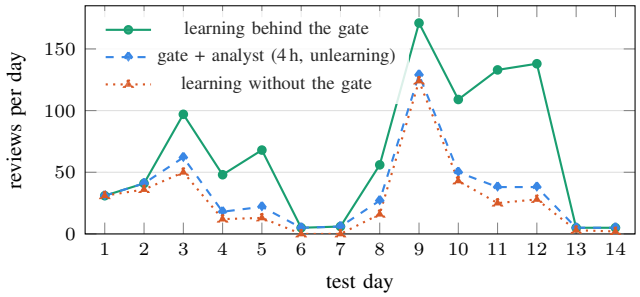


Fig. 2. Analyst reviews per day on DEDALE's host logs over the 14 test days (attack in the first week). Without learning the workload is off this scale (Table III). The ungated store's lower load is the attacker learned as routine (C3).

TABLE III
LEARNING MODES ON DEDALE HOST LOGS (14 TEST DAYS).

Mode	Reviews/day	Attack cases	Events alerted	Attack learned
static	6,095.4	80	473	0
frozen	65.2	72	400	0
gated	65.2	72	400	0
ungated	27.4	38	99	38

C. Memory poisoning through the analyst (C3)

Analyst verdicts bypass the gate by design. A simulated analyst missing 20% of attacks taught the store 20 attack cases' patterns, and only 256 of 400 attack events still raised alerts (Table IV). Unlearning on confirmed compromise removes the effect: with the same mistakes, nothing was learned and all 400 events alerted. It costs 10% more reviews with a perfect analyst (30.9 to 34.1 per day) and makes false alarms more expensive (31.4 to 44.6 per day at 5% false alarms), because a false confirmation stops learning on a healthy host.

D. What the LLM adds (C4)

With NCE, reviews fall from 65.2 to 54.8 per day (16%), by dismissing cases whose hypotheses are all structurally infeasible. Under a closed-world check this dismissed 2 of

TABLE IV
ANALYST MISTAKES, WITHOUT / WITH UNLEARNING (HOST LOGS, 4 H DELAY).

Analyst	Reviews/day	Attack learned	Events alerted
always right	30.9 / 34.1	0 / 0	400 / 400
misses 5%	30.6 / 34.1	8 / 0	397 / 400
misses 20%	30.1 / 34.1	20 / 0	256 / 400
5% false alarms	31.4 / 44.6	0 / 0	400 / 400

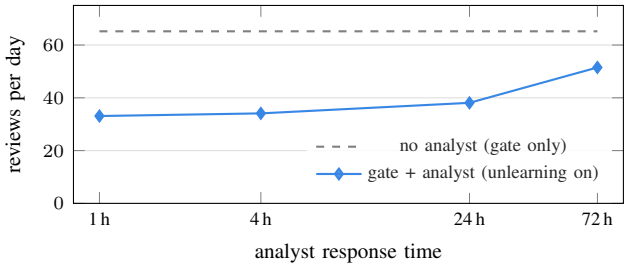


Fig. 3. Analyst response time sets the workload (host logs): benign patterns are learned only when the verdict arrives. Detection is unchanged (all 400 attack events alert at every delay), and on the network the attacker's server was contained exactly one response time after its first alert.

the 72 attack cases: the attacker copied data from a server the store has no telemetry for, and "no record of access" was read as "impossible". Treating hosts without telemetry as unknown removed both misses at no cost (89 benign dismissals either way). With an analyst in the loop, NCE adds little (33.2 reviews per day with NCE, 34.1 without, both with unlearning).

E. Operating the loop (C5)

Detection does not depend on how fast analysts answer, but workload does: host reviews per day rise from 33.1 to 51.5 as the delay grows from 1 h to 72 h, because benign patterns are learned only when their verdict arrives. On the network, the attacker's server stays reachable until the verdict: containment came 60 min to 72 h after the first alert, equal to the delay (Fig. 3).

VII. DISCUSSION AND LIMITATIONS

Where the defence lives. The LLM itself was not made robust. Acting alone, it was hijacked on 29.6% of the GUIDE pairs, and on DEDALE it named the steered host in 30–32 of 38 cases. What held was the structure around it. Hypotheses are checked against the trusted fields and the environment model before any response is ranked. A target the trusted fields do not support is flagged, not acted on. Only structured fields are ever learned. This is the principle of recent designs that keep untrusted data out of an agent's control flow [11], applied to a SOC: the LLM's output is advisory, and what it may cause is decided by components it cannot write to. Prompt-level defences [9], [10] reduce how often a model follows injected text; our measurements concern what happens when it does.

Memory is the larger surface. A system that learns what is normal is a classic poisoning target [12], and agents' memories and knowledge bases have the same weakness [13], [14]. On DEDALE, learning without the gate cut the labelled attack events that raised alerts from 400 to 99; with the gate, all 400 still did. The weaker route is the analyst. Verdicts bypass the gate by design, and with a simulated analyst who missed 20% of the attacks, only 256 of the 400 still raised alerts. Unlearning on confirmed compromise closed this route: a targeted form of machine unlearning [15] that retracts what was recently learned on a host once the host is confirmed compromised. It has two costs. It works only because each attacked host received at least one correct malicious verdict. And it makes false confirmations expensive, raising reviews from 31.4 to 44.6 per day at 5% false alarms. The prototype lets a lead withdraw a wrong confirmation, which restores what it retracted; we did not evaluate this.

Absence of telemetry is not impossibility. The closed-world feasibility check dismissed 2 real attack cases, because the store had no telemetry from the server the attacker copied data from. Any structural check of LLM hypotheses inherits the coverage of its environment model. Hosts without telemetry must be treated as unknown, and a deployment needs to know which hosts it cannot see.

Workload. Most of the reduction comes from learning: 6,095 to 65 reviews per day. Analyst verdicts feeding the store take it to 34. The LLM cuts reviews by 16% on its own, and by little once an analyst is in the loop (34.1 to 33.2). On this data, the LLM's measurable effect on workload is modest. We did not measure its effect on how fast or how accurately analysts decide, which is where its explanations would matter.

Limitations.

- One real dataset. Labels exist for two attacked Windows clients, and the network results rest on few malicious verdicts.
- A simulated analyst: an oracle with random, per-case mistakes. Real analysts' mistakes are likely correlated with how well the attacker blends in, which would make C3 harder.
- One LLM on DEDALE (`gemini-3.1-flash-lite`). The second model, `gpt-oss-20b`, was run on the GUIDE development set only (Table I).
- GUIDE v2's entities, evidence and payloads are synthetic. E1 injects text only into command lines, with three payload families.
- Tampering with structured sensor fields is outside the defences (Section VI-A).
- All containment is simulated.
- The prototype's review-queue order was tuned on DEDALE and is not evaluated.

What a deployment would need:

- an asset inventory and environment model that are kept current, since SSE and entity binding are only as good as they are;
- known telemetry coverage per host;

- a study with real analysts, who face alert fatigue [5];
- a second dataset, to check that the gate's settings transfer (a 24 h delay, and a hold while related cases are escalated).

VIII. Conclusion

An LLM-driven SOC can be attacked through what its model reads and through what its memory learns. In our pipeline, the model was easy to steer, but its output was advisory: hypotheses were checked against trusted fields and the environment model, steered targets were flagged, and no misdirected action ran. The memory was the larger risk. Learning made the workload manageable (6,095 to 65 reviews per day on DEDALE's host logs) and, without a gate, also learned the attacker. The gate held, and the weakest remaining route, wrong analyst verdicts, was closed by retracting what was learned on a host once it was confirmed compromised. A deployment would need a maintained environment model, known telemetry coverage, and evaluation with real analysts and more than one real dataset. The pipeline, the demo and the scripts that rebuild every number are available [TODO: repository link].

References

[1] J. Roy and S. K. Singh, "AgentSOC: A multi-layer agentic AI framework for security operations automation," in *Proc. 5th IEEE Int. Conf. on AI in Cybersecurity (ICAIC)*, Houston, TX, USA, 2026, arXiv:2604.20134.

[2] S. Freitas, J. Kalajdjieski, A. Gharib, and R. McCann, "AI-driven guided response for security operation centers with Microsoft Copilot for Security," arXiv:2407.09017, 2024.

[3] M. Lanvin and F. Majorczyk, "DEDALE: Dataset for evaluating detection of APT among logs and events," Recherche Data Gouv, 2026, doi:10.57745/Y5JLDG.

[4] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Tech. Rep., 2018.

[5] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Security Symp.*, 2022.

[6] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS ML Safety Workshop*, 2022, arXiv:2211.09527.

[7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[8] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symp.*, 2024.

[9] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," arXiv:2403.14720, 2024.

[10] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proc. 34th USENIX Security Symp.*, 2025.

[11] E. Debenedetti *et al.*, "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[12] B. I. P. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S. Lau, S. Rao, N. Taft, and J. D. Tygar, "ANTIDOTE: Understanding and defending against poisoning of anomaly detectors," in *Proc. 9th ACM SIGCOMM Internet Measurement Conf. (IMC)*, 2009.

[13] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[14] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proc. 34th USENIX Security Symp.*, 2025.

[15] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2015.